

G7 · 并发协议与 C++ 内存模型

Professional Handbook · 阅读试件

G7

v1.1.1

ACCEPTED CONTENT / PILOT PDF

权威内容来源为 Markdown。本件仅供阅读与评审，未获正式发布批准。

阅读与版本说明

PREVIEW / DRAFT · 非权威阅读视图

本件从已接受的 Markdown 冻结提交取源。正文中的“待审核”保留提交时的历史文字；内容接受不等于 PDF 或正式发布批准。标题、章节编号、代码与 Gate 不因出版改写。封面、目录和代码续页标签是生成的阅读辅助。

制品标识 G7

版本 1.1.1

状态 ACCEPTED CONTENT / PILOT PDF

阅读密度 balanced · Visual Profile Candidate，尚未冻结

基准 commit 8f479deaf660533b2ad82e1f721eb41a363112b6

准备标识 c0ce3381c728be48e0f7cc67

使用方式 目录与书签可跳转；合订版中的核心文档链接跳转到本件对应篇章。独立版在源字节匹配时提供固定 commit 链接，否则指向随附的 source 快照。完整输入以准备记录为准，可能包含未提交变更；正文原有编号不重新连续编号。

G7 v1.1.1 ACCEPTED CONTENT / PILOT PDF

目录

阅读与版本说明	2
G7 · 并发协议与 C++ 内存模型	4
阅读入口	4
1. 阅读模型	5
2. 数据竞争与语言内存模型	5
3. 顺序与同步关系	7
4. 原子性与内存序	9
5. 互斥锁与条件变量	12
6. 原子状态转换与 CAS	15
7. ABA、进展保证与安全回收	16
8. 并发队列的交接协议	19
9. 线程生命周期与停机	21
10. 任务执行、线程池与背压	24
11. 并发架构与写入权限	26
12. 跨层关系图	29
13. 统一并发审查	29
14. 并发反模式	31
15. 工程原则回查	33
16. 实验与验证	34
17. Final Gate	38
18. Final Gate · 参考答案与常见误判	41
19. G0 ~ G7 的统一模型	42
20. 实际代码的跨层审查	43
21. 本章范围与后续阅读	44
22. 参考与验证入口	44

G7 · 并发协议与 C++ 内存模型

SOURCE SNAPSHOT CONTENT · 8f479de

以下状态文字保留冻结 Markdown 当时的历史陈述。当前文件是后续生成的阅读试件，不改变该历史状态，也不表示正式发布获批。

版本：1.1.1 · Professional Handbook · 全书一致性修订

状态：本轮编辑修订待集中审核；接受历史与冻结候选见系列状态。PDF **NOT BUILT / NOT VALIDATED**。

语言基线与范围：C++23。前置为 G0 ~ G6；从数据竞争、同步和回收到线程停机与并发架构；工程论证不等于形式化证明。

阅读约定：Editorial Profile v1.0 · 全书术语、证据与引用。

上一章：G6 · 全系列导航 · 下一章：G8

阅读入口

本章先区分数据竞争（data race）、原子性（atomicity）和不变量（invariant），再建立顺序与生命周期协议。先发生关系（happens-before, HB）指语言允许依赖的顺序关系，不是墙钟上的先后。首次阅读第 1 ~ 11 节；跨层关系图、审查和反模式用于回查。读写改操作（read-modify-write, RMW）、比较并交换（compare-and-exchange, CAS）都是实现协议的手段，不是设计的出发点。

章节目录

CHAPTER READING PATH · 原文章内导航

- 1. 阅读模型
- 2. 数据竞争与语言内存模型
- 3. 顺序与同步关系
- 4. 原子性与内存序
- 5. 互斥锁与条件变量
- 6. 原子状态转换与 CAS
- 7. ABA、进展保证与安全回收
- 8. 并发队列的交接协议
- 9. 线程生命周期与停机
- 10. 任务执行、线程池与背压
- 11. 并发架构与写入权限
- 12. 跨层关系图
- 13. 统一并发审查
- 14. 并发反模式

- 15. 工程原则回查
- 16. 实验与验证
- 17. Final Gate
- 18. Final Gate · 参考答案与常见误判
- 19. G0 ~ G7 的统一模型
- 20. 实际代码的跨层审查
- 21. 本章范围与后续阅读
- 22. 参考与验证入口

1. 阅读模型

1.1 G7 到底解决什么问题？

G6 讨论缓存、地址翻译和一致性如何影响成本；本章讨论两个线程何时可以合法访问并观察同一状态。这是语言模型问题，不能由“处理器最终会把缓存同步”代替。

工程上应先确定对象存活、写入权限与跨线程交接，再选择 `atomic`、`mutex` 或队列。协议若没有正确的同步和回收关系，某个平台上长期运行正常也不是其合法性的证明。

1.2 G7 的统一链路

本章的推理顺序是：内存位置与冲突访问 → 数据竞争检查 → 原子性及顺序关系 → 对象所有权和生命周期 → 并发协议、进展及容量约束。原子 API 只解决链条中的一部分。

先写清谁拥有数据、谁能修改、哪些字段必须保持一致、何时交接权限及哪些操作需要 HB；之后才选同步原语。队列、线程池和快照都应回到这组问题，而不是从某种“更高级的无锁结构”倒推业务模型。

2. 数据竞争与语言内存模型

2.1 Data Race 是 G7 的第一道门槛

考虑：

[反例片段 · 未定义行为：无同步的冲突访问；不要用于生产代码]

```
int value = 0;
bool ready = false;

// Thread A
value = 42;
ready = true;

// Thread B
while (!ready) {
}

use(value);
```

人类直觉：

```
Thread B 看到了 ready == true
      ↓
Thread A 肯定已经执行了 value = 42
      ↓
所以 value == 42
```

C++ 并不允许这样推理。这里首先存在：**Data Race** 因此：**Undefined Behavior**

2.2 Data Race 的核心条件

数据竞争（data race）的判断对象是潜在并发的冲突操作：修改同一内存位置，或者开始/结束与访问重叠对象的生命周期，都可能形成冲突；至少一方非原子，且没有所需的 happens-before 次序时，会进入未定义行为。只读且生命周期稳定的同一对象通常不构成数据竞争。不同成员是否对应独立内存位置还需留意位域等情况。这个规则不是“读到旧值”的概率模型，更不能用在某次机器运行中恰好没出错来否定。规则：[N4950 intro.races](#)

2.3 read/read 与 read/write

通常：

```
read / read
→ 不冲突
```

而：

```
read / write
write / write
→ conflicting
```

所以：

```
const Config config;
```

在正确 publication 后被多个 threads 只读：是非常自然的模型。真正复杂的是：Shared Mutable State。

2.4 Data Race 不等于 Race Condition

数据竞争（data race）采用 §2.2 的语言判据；竞态条件（race condition）则是更广义的逻辑问题：结果依赖不受控制的交错。前者涉及未定义行为，后者即使只使用原子操作也可能破坏业务不变量。例如：

[机制片段 · 不承诺独立编译]

```
std::atomic<int> balance{100};

if (balance.load() >= 80) {
    balance.fetch_sub(80);
}
```

两个 threads 都可能：

load 100、load 100、subtract、subtract。

最后：-60 所有 atomic operations 本身合法，但业务 invariant 被破坏。所以：

Data-race-free $\neq$ Race-condition-free

2.5 volatile 不是并发同步

C++ volatile 不提供原子性、线程间同步或 happens-before。把共享 ready 标志改成 volatile，不能修复普通 payload 的发布协议。

[反例片段 · 未定义行为风险：volatile 不建立线程同步]

```
volatile bool ready = false;
```

它与 atomic、mutex 承担不同语义责任；应使用适合协议的同步机制，而不是借 volatile 阻止某种优化来模拟线程通信。

3. 顺序与同步关系

3.1 sequenced-before

描述：同一 thread 内 C++ abstract machine 的顺序关系。例如：

[机制片段 · 不承诺独立编译]

```
data = 42;
ready.store(true);
```

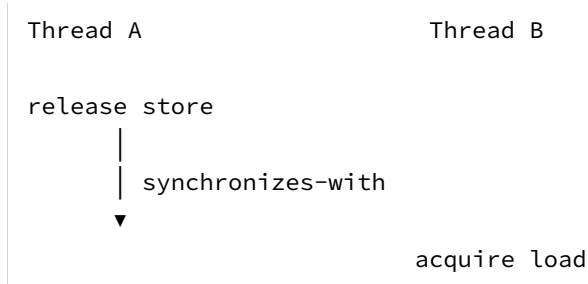
有：

```
data = 42
  |
  | sequenced-before
  ▼
ready.store(true)
```

注意：这不是“CPU 一定先把 data 写入 DRAM”。它是：语言级 ordering relation。

3.2 synchronizes-with

跨线程 synchronization primitive 可以建立：**synchronizes-with** 例如 release/acquire：



前提是：acquire 真正接到了对应的 publication。不是只因为源码中写了两个关键词。

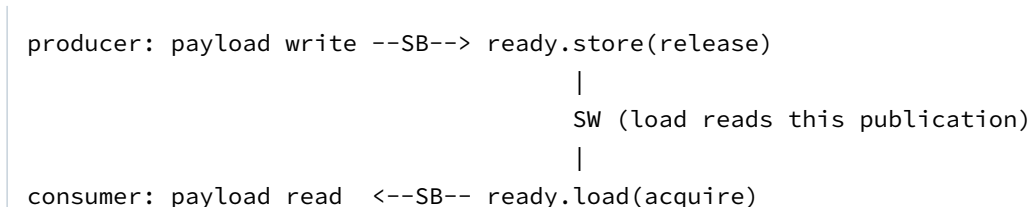
3.3 happens-before

happens-before 是建立合法跨线程访问的重要关系。在本章不使用 consume 的协议中，可以沿线程内 sequenced-before 与跨线程 synchronizes-with 组成传递链。必须逐条说明同步边由哪一次实际观察建立，而不是只标注两端使用了 acquire/release。

如果普通 payload 的冲突访问之间存在所需 HB，且对象生命周期和其余访问也合法，就可以推理这次读写。单条 HB 边不等于整个系统无数据竞争，更不决定所有无关原子操作的全局顺序。

3.4 最重要的一张图

单次发布的关键不是时间先后，而是可组合的关系。生产者写 payload，随后 release-store 标志；消费者的 acquire-load 必须读到该 store（或它引领的适用 release sequence）的值，才能建立 synchronizes-with。结合线程内 sequenced-before，payload 写入 happens-before 消费者读取。



G7-D1 仅做一次发布，不重置标志，也不在发布后并发修改 payload。若复用同一槽位，还需消费者完成读取到生产者下一次写入的反向交接；看到 true 不自动授予永久读取许可。

3.5 为什么 data 可以不是 Atomic ?

普通 payload 不必全部改为 atomic：在上述单次发布协议中，写入与读取已由 release/acquire 的 HB 链排序。原子标志承担交接责任，payload 在交接后不再被生产者修改。

这个结论依赖完整协议，不是“有一个 atomic 就保护附近变量”。若另有写者、重置标志后立即覆盖、或对象在读者使用前已销毁，原来的论证必须重做。

4. 原子性与内存序

4.1 Atomicity ≠ Ordering

原子性 (atomicity) 描述某个访问或 RMW 作为不可分割的操作被观察；内存序 (memory ordering) 说明它与其他相关操作建立什么顺序约束。原子对象本身的操作合法，不自动把一组业务操作合成事务。

例如对余额先 load 再 store，两步各自原子却可能丢失并发更新。需要根据不变量选择互斥区或条件 RMW，而不是把每个字段换成 atomic 就结束审查。

4.2 Atomic Operation 三类

Load

```
x.load(order);
```

Store

```
x.store(value, order);
```

Read-Modify-Write

[机制片段 · 不承诺独立编译]

```
x.fetch_add(...);  
x.exchange(...);  
x.compare_exchange_weak(...);
```

同时有：

read side + write side

4.3 Memory Order 属于 Operation

不是：“这个 atomic variable 是 acquire atomic”而是：

[机制片段 · 不承诺独立编译]

```
x.load(std::memory_order_acquire);  
  
x.store(  
    value,  
    std::memory_order_release);  
  
x.fetch_add(  
    1,  
    std::memory_order_relaxed);
```

同一个 atomic object：不同 operations 可以使用不同 memory order。

4.4 memory_order_relaxed

保证：

atomicity + 该 atomic object 自己的 modification order

但不自动建立：surrounding ordinary data 的跨线程 synchronization。典型：

[机制片段 · 不承诺独立编译]

```
processed.fetch_add(
    1,
    std::memory_order_relaxed);
```

适合：

statistics、independent counter、ticket generation。

前提：不依赖它去发布其它 state。

4.5 Modification Order

每个 atomic object 都拥有：对其所有 modifications 的一致总顺序。例如：

```
x:
0
↓
1
↓
2
↓
3
```

所以 relaxed 并不是：“atomic value 本身也完全乱序。”它仍然具有该 object 自己的 atomic coherence。

4.6 memory_order_release

典型角色：**Publish**

[机制片段 · 不承诺独立编译]

```
payload = build_payload();

ready.store(
    true,
    std::memory_order_release);
```

表示：release 之前 sequenced-before 的相关 operations，可以通过合适 acquire 建立跨线程 HB。

不要翻译成：flush cache to RAM

4.7 memory_order_acquire

典型：

[机制片段 · 不承诺独立编译]

```
if (ready.load(
    std::memory_order_acquire)) {
    use(payload);
}
```

如果 acquire 观察到对应 release publication，则 release 之前的 writes：HB acquire 之后的 reads。

4.8 Release / Acquire 的核心不是 Cache Flush

release/acquire 是语言排序合同，不是“先把所有缓存刷到 DRAM，再让另一核从 DRAM 重读”。编译器针对目标架构把所需约束映射为适当指令和屏障；硬件也可能直接在缓存之间传递数据。

因此协议正确性在语言关系层证明，成本才在目标代码与机器层测量。二者分开，才能避免因某架构的强顺序或某次汇编看起来简单，就削弱可移植程序的必要关系。

4.9 memory_order_acq_rel

主要用于 RMW：

Acquire previous publication + Publish new state

例如：

[机制片段 · 不承诺独立编译]

```
state.compare_exchange_weak(
    expected,
    desired,
    std::memory_order_acq_rel);
```

但：**RMW 不自动意味着一定需要 acq_rel**。如果只需要 atomicity：relaxed 可能就够。如果只需要 acquire：acquire 也可能够。

4.10 memory_order_seq_cst

memory_order_seq_cst 除相应 acquire/release 作用外，还对 seq_cst 操作（包括相应 fence）提供符合标准约束的单一总序。它不是把所有非原子访问、混合弱序操作和外部事件都排成一个全局时钟。选择 seq_cst 可以简化部分推理，但不能补救对象已销毁、复合不变量被拆散或存在非原子数据竞争的问题。混合内存序时仍需逐条说明同步关系，而不是引用“最强”两个字。规则：N4950 atomics.order

4.11 Memory Order 不是性能等级表

内存序不是从 relaxed 到 seq_cst 的固定性能排行榜。目标架构、具体操作、编译结果和争用都会改变成本；相同关键字在不同场景中不保证相同开销。

选择顺序应是业务不变量、所需 HB、同步协议、足够的内存序，最后才是测量。无证据地降序可能破坏正确性；无分析地全部升到 seq_cst 也不能弥补生命周期或进展缺陷。

5. 互斥锁与条件变量

5.1 Mutex 保护的是 Invariant

不是：“锁住一个变量”。例如：

[机制片段 · 不承诺独立编译]

```
struct Account {
    int balance;
    int reserved;
};
```

真正需要保护的是：

$balance \geq 0$ 、 $reserved \geq 0$ 、 $reserved \leq balance$ 。

也就是：多个字段构成的 logical invariant。

5.2 Mutex 提供两件事

Mutual Exclusion

同一时刻：只有一个 owner 进入 critical section。

Memory Synchronization

一个 thread：

```
writes
↓
unlock
```

另一个：

```
successful lock
↓
reads
```

可以建立相应 synchronization / HB。因此 mutex 内保护的数据：

```
int value_;
```

不需要全部改 atomic。

5.3 RAII Locking

默认：

```
std::lock_guard lock{mutex};
```

需要：

unlock/relock、condition_variable、deferred locking。

时使用：

```
std::unique_lock lock{mutex};
```

多个 mutex：

```
std::scoped_lock lock{a, b};
```

优先避免手写：

[机制片段 · 不承诺独立编译]

```
mutex.lock();  
...  
mutex.unlock();
```

5.4 Critical Section 原则

不是：“越短越好。”而是：覆盖维护 **invariant** 所需的最小完整 **logical transaction**。不能为了缩短锁：

check、unlock、...、lock、update。

把本应 **atomic** 的业务操作拆开。同时不要无必要地把：

I/O、network、sleep、large computation、callbacks。

放在 hot shared lock 内。

5.5 Deadlock

经典：

- Thread A:
- holds A
- waits B
- Thread B:
- holds B
- waits A

解决方法：

- consistent lock order
- std::scoped_lock
- architecture redesign

如果系统出现：

大量 **mutex**、随机组合获取。

应该考虑：是否 shared-state topology 本身已经过度复杂。

5.6 Condition Variable 等待的是 Predicate

不是 Notification。正确：

[机制片段 · 不承诺独立编译]

```
cv.wait(lock, [&] {
    return closed_ || !queue_.empty();
});
```

真正业务事实：

queue non-empty、or、closed。

notify_one() 只是：“state 可能已经改变，请重新检查。”

5.7 Notification 是 Edge，Predicate 是 State

条件变量（condition variable）不保存一张可消费的通知清单。正确性依赖受同一互斥协议保护的谓词：生产者持锁改变状态；消费者持锁检查状态，并通过 wait 原子地释放锁进入等待。唤醒后重新持锁检查谓词。若通知早于消费者检查，持久状态使它无需等待；若消费者需要等待，锁与 wait 的协议避免“检查完但尚未等待”的空窗丢失状态改变。仅把标志改成 atomic，再在锁外修改并通知，不自动提供这个保证。G7-D2 把队列、closed 和等待条件放在同一把 mutex 下。

5.8 Spurious Wakeup

wait() 可以在没有目标业务事件时返回。另外多个 waiters 竞争：

A wakes and consumes item、B wakes afterwards、queue empty。

因此：

[机制片段 · 不承诺独立编译]

```
if (!predicate()) {
    cv.wait(lock);
}
```

通常错误。应：

```
cv.wait(lock, predicate);
```

或：

[机制片段 · 不承诺独立编译]

```
while (!predicate()) {
    cv.wait(lock);
}
```

6. 原子状态转换与 CAS

6.1 为什么 load + store 不等于 Atomic Transition ?

[机制片段 · 不承诺独立编译]

```
if (state.load() == Ready) {  
    state.store(Running);  
}
```

两个 individually atomic operations, 仍然可能:

Thread A load Ready、Thread B load Ready、Thread A store Running、Thread B store Running。

两个 threads 都认为自己抢到了 transition。

6.2 CAS

[机制片段 · 不承诺独立编译]

```
value.compare_exchange_strong(  
    expected,  
    desired);
```

语义:

- if current == expected:
- current = desired
- return true
- else:
- expected = current
- return false

关键: CAS 把 check + update 合成一个 atomic RMW。

6.3 expected 是 In/Out Parameter

成功:

atomic = desired、expected unchanged、return true。

失败:

atomic unchanged、expected = actual observed value、return false。

这就是为什么 CAS loop 不需要每次重新 load。

6.4 CAS Loop

```

load current
↓
compute desired
↓
CAS
├ success → done
└ failure
    ↓
    current updated
    ↓
    retry

```

这是：**Optimistic Concurrency** 观察:current state 尝试:conditional commit 冲突:retry

6.5 Weak vs Strong

`compare_exchange_weak` 允许伪失败，适合本来就要重试的循环；`compare_exchange_strong` 不允许这种伪失败，适合需要解释单次失败含义的用法。两者仍须处理值确实改变的情况。

`weak/strong` 描述 CAS 的失败模型，`relaxed/acquire/release` 描述排序与同步，不能互相替代。循环中还要根据更新后的 `expected` 重算 `desired`，并检查进展和重复副作用。

6.6 Success / Failure Ordering

双内存序 CAS 必须分别解释成功的 `read-modify-write` 和失败的 `load`。失败不执行写入，因此 `failure order` 不得是 `release` 或 `acq_rel`；应按失败后是否读取发布数据选择允许的顺序，不能把成功侧的 `release` 要求机械复制过来。

[机制片段 · 不承诺独立编译]

```

while (!state.compare_exchange_weak(expected, desired,
                                     std::memory_order_acq_rel,
                                     std::memory_order_acquire)) {
    desired = next(expected); // 依据更新后的 expected 重算。
}

```

`next` 必须与实际状态转换合同匹配，循环内也不能不经处理地重复外部副作用。`weak CAS` 的伪失败与 `memory order` 的强弱是两个问题。

7. ABA、进展保证与安全回收

7.1 CAS 只比较 Current Value

CAS 比较当前值表示 (value representation)，不是调用用户定义的 `operator==`，也不记录对象经历过的历史。指针或索引可能从 A 变为 B 再回到相同表示 A；CAS 看到相等仍无法判断它是否属于同一逻辑代次，这就是 ABA。需要历史身份时，可以把 `generation/tag` 纳入被比较状态；还要考虑宽度、回绕和原子实现条件。`tag` 并不保护已经取得的指针所指对象，内存回收协议仍是独立责任。

7.2 ABA 不是 Use-after-free

ABA: state/version/history 问题。Reclamation: object lifetime 问题。两者经常同时出现，但必须分开。

7.3 Unlinked $\neq$ Reclaimable

一个 node:

```

Linked
↓
Unlinked
↓
Retired
↓
Reclaimable
↓
Destroyed

```

从 lock-free structure 中删除: 只解决 Logical Membership。不代表: 没有 reader 仍持有 pointer。

7.4 Pointer Acquisition $\neq$ Safe Dereference

并发:

```
Node* p = head.load();
```

只得到: pointer value。并没有自动保证: *p object lifetime 仍然 active。另一个 thread 可能: remove、delete。

发生在: load p 和: p->next 之间。

7.5 Tagged Pointer / Generation

把: pointer 扩展成: (pointer, generation) 例如:

```

(A, 41)
→
(B, 42)
→
(A, 43)

```

旧: (A, 41) 不再等于: (A, 43) 可以检测一类 ABA。但: 不自动解决 pointee lifetime。

7.6 Reclamation Strategies

Reference Counting

```

reader holds ownership
→ node cannot die

```

简单组合, 代价:

atomic refcount、control block、coherence traffic。

Hazard Pointer

reader 显式发布: I currently protect Node A reclaimer:

```
A appears in hazard set
→ cannot free
```

流程:

```
load pointer
↓
publish hazard
↓
revalidate source
↓
dereference
↓
clear hazard
```

Epoch-based Reclamation

reader 表示: I am active in epoch E retired node: 等所有可能仍在旧 epoch 的 readers 离开后再 reclaim。

RCU / QSBR

```
build new generation
↓
publish
↓
wait grace period
↓
reclaim old generation
```

特别适合:

read-heavy、write-rare。

系统。

7.7 Lock-free 不是性能等级

Lock-free 是: Progress Guarantee。不是: “一定比 mutex 快”。CAS contention 可能产生: retry storm、cache-line ping-pong。

一个低 contention mutex 可能更快、更简单。

7.8 Progress Guarantees

粗略:

Wait-free、 $\Rightarrow$ 、Lock-free、 $\Rightarrow$ 、Obstruction-free。

Lock-free

系统整体保证：持续有某个 operation 取得进展。不保证每个 thread 都不饿死。

Wait-free

更强：每个 operation 都在有限步骤内完成。

7.9 atomic<T>::is_lock_free() 不是算法证明

它只回答：这个 atomic object 的 implementation 是否 lock-free。不代表：整个 data structure / allocator / reclamation / operation path 是 lock-free。

8. 并发队列的交接协议

8.1 Queue 首先按 Topology 分类

Queue	Producer	Consumer
SPSC	1	1
MPSC	N	1
SPMC	1	N
MPMC	N	N

Topology 直接决定：

- 谁写 enqueue cursor?
- 谁写 dequeue cursor?
- 哪里需要 CAS?
- 哪里可能 single-writer?

8.2 SPSC 的关键简化

- Producer:
- only writer of tail
- Consumer:
- only writer of head

因此：不需要 CAS 去争抢 cursor ownership。只需正确 publication。

8.3 SPSC 的两条 HB Edge

Producer → Consumer

```

construct slot
↓
tail.store(release)
↓
tail.load(acquire)
↓
consume slot

```

因此：

slot write、HB、slot read。

Consumer → Producer

```

finish consuming
↓
head.store(release)
↓
head.load(acquire)
↓
reuse slot

```

因此：

old generation use、HB、new generation construction。

这两条就是 SPSC correctness 骨架。

8.4 MPSC 的关键新问题

多个 producers 可以：reserve slots 但 reservation completion 可以 out-of-order：

```

P0 reserves slot 10
pause

P1 reserves slot 11
writes + publishes

```

于是：

reservation frontier $\neq$ ready frontier

简单 global tail 不够。

8.5 MPMC 为什么需要 Per-slot Generation

常见有界 MPMC 环形队列用每槽 sequence/generation 区分空闲、已占位、已发布及可复用的代次。多生产者的 reservation 不等于元素构造和发布已经完成；消费者必须按设计好的槽状态取得元素。这不是所有 MPMC 算法都必须采用同一种布局的定理。序号回绕、失败路径、对象构造/析构及消费者回收都需要单独证明。本章解释协议问题，不把一个 per-slot 数字当作完整队列正确性证明。

8.6 Bounded Queue = Backpressure

如果 producer rate:

、consumer rate。

无界 queue:

memory ↑
latency ↑

bounded queue: full 显式告诉上游: 系统已达到当前 processing capacity。Full 是: 正常系统状态。不是异常。

8.7 Queue 是 Ownership Boundary

成功:

```
queue.push(std::move(job));
```

自然语义:

```
Producer owns Job
↓
Queue owns Job
↓
Consumer owns Job
```

因此跨线程 queue 中优先:

owned value、move-only handle、lease、unique ownership。

而不是未经约束的 borrowed pointer。

9. 线程生命周期与停机

9.1 std::thread

如果 destructor 时: joinable == true 会: std::terminate()。所以 thread lifetime 必须显式解决:

join、or、detach。

9.2 detach() 不解决 Lifetime

它只是: 解除 std::thread object 与执行线程的 join relationship。不会解决:

thread still accesses this、thread still accesses queue、thread still accesses stack object。

所以 detach 常常只是: 隐藏 ownership 问题。

9.3 `std::jthread`

核心:

RAII join + cooperative stop

Destructor 大体:

```
request_stop
↓
join
```

但: stop 是 request, 不是 kill。Worker 必须 cooperate。

9.4 `stop_token`

表示: 观察 cancellation request 的 capability。不是: pause/resume state 而是 monotonic:

```
not requested
↓
requested
```

9.5 Cancellation 必须唤醒 Blocked Thread

合作取消必须同时解决请求可见、阻塞可唤醒和退出后可 join。`request_stop()` 只提出请求, 不会强制终止任意 wait、I/O 或持锁操作。

一种方案是在同一 mutex 下设置 closed 等终止谓词并通知所有等待方; 另一种是使用 `condition_variable_any` 提供的 stop-token 等待重载。仅注册一个在锁外调用 `notify_all` 的 stop callback, 仍可能遗漏检查与等待之间的竞争窗口。普通条件变量实验 G7-D2 使用显式 close; 它不冒充 stop-token 重载的验证。规则: N4950 `condition_variable_any`

9.6 Cancellation Point 必须保持 Invariant

不能:

```
update half of shared state
↓
see stop
↓
return
```

如果状态因此不一致。Cancellation 和 Exception Safety 有同样结构:

```
prepare
↓
safe cancellation point
↓
commit
```

9.7 Member Destruction Order

成员按声明的逆序析构，因此让工作线程成员最后声明，可以使它先于其访问的依赖成员析构。但声明顺序只是前提之一：线程必须收到可执行的停止协议，被阻塞时能唤醒，且 `join` 不与被持有的锁形成死锁。`jthread` 析构在仍为 `joinable` 时请求停止并 `join`，却不能替代用户协议。若工作函数不响应停止、继续等待外部资源，或者回调重新进入已在析构的 `owner`，仅调整成员顺序并不能解决问题。

9.8 Shutdown 基本顺序

1. 停止接收新工作。
2. 发布关闭或停止状态。
3. 唤醒阻塞线程。
4. 按合同排空已接收工作，或取消并处理未完成工作。
5. 等待线程函数退出。
6. 完成 `join`。
7. 销毁共享状态及其资源。

具体依赖可以要求更细的步骤，但不得在仍有访问者时先销毁依赖。

9.9 Drain vs Abort

Drain

```
reject new work
↓
process already accepted work
↓
exit
```

Abort

```
reject new work
↓
request cancellation
↓
discard/cancel according to contract
↓
exit
```

两者绝不能用一个模糊：

`stop();`

让 `caller` 猜。

10. 任务执行、线程池与背压

10.1 Thread ≠ Task

Thread: execution resource. Task: unit of work. Thread pool 的本质:

```
many tasks
↓
bounded worker threads
```

把: logical concurrency 和: physical parallelism 解耦。

10.2 C++23 Task Representation

非常适合:

[机制片段 · 不承诺独立编译]

```
using Task =
    std::move_only_function<void()>;
```

因为 Task 经常捕获:

unique_ptr、socket、buffer lease、move-only handle。

这和 ownership transfer 模型一致:

```
Submitter
→ Queue
→ Worker
```

10.3 Worker 不能持 Queue Lock 执行 Task

正确:

```
lock queue
↓
move task out
↓
unlock
↓
execute task
```

Queue mutex 只保护: queue invariant. Task execution 已经变成: worker-local ownership。

10.4 Pool 必须考虑两种 Bound

Worker count 限制: physical parallelism. Queue capacity 限制: backlog。只有 bounded workers、unbounded queue: overload 仍然可以把 memory/latency 推爆。

10.5 CPU-bound 与 I/O-bound Pool

CPU-bound:

```
worker count
≈ available compute resources
```

从此附近开始 benchmark。I/O-bound: threads 可能大量 blocked, worker count 可能更大。但更成熟可能使用: async I/O 避免大量 blocked OS threads。

10.6 Resource Isolation

不要把:

CPU tasks、blocking database calls、fsync、network waits、background cleanup。

全部塞进一个 global pool。否则: blocking work 会 starve CPU work。典型:

CPU pool、blocking pool、async I/O runtime。

分离。

10.7 Thread Pool Starvation Deadlock

Pool 只有一个 worker:

```
Task A runs
↓
submit Task B to same pool
↓
A waits future B
↓
B waits for a worker
↓
the only worker is occupied by A
```

没有 mutex cycle, 仍然 deadlock。这是: execution-capacity dependency cycle。

10.8 Bounded Submit 也能 Deadlock

多个 workers:

```
each running task
↓
blocking submit child task
```

queue full。所有 workers: wait queue capacity 但 queue capacity 只能通过: worker consuming tasks 来释放。于是:

```

Workers
↓ wait for
Queue Capacity
↓ requires
Workers

```

形成 cycle。Backpressure 必须结合：resource graph 分析。

10.9 Global Queue vs Work Stealing

Global MPMC:

simple、good natural balancing、central contention。

Per-worker queue:

locality、less global contention。

空闲 worker: steals Work stealing 本质: common case local ownership, rare case cross-worker coordination。

11. 并发架构与写入权限

11.1 并发设计最重要的问题：有几个 Writer ?

不是：有几个线程。而是：

- 每一份 mutable state
- 有几个 writer?

复杂度通常随着：multiple writers 急剧上升。

11.2 Reasoning Complexity Ladder

粗略：

```

Thread-local mutable
      ↓
Single-writer mutable
      ↓
Immutable shared
      ↓
Ownership transfer
      ↓
Mutex-protected shared mutable
      ↓
Atomic shared mutable
      ↓
Multi-object lock-free shared mutable

```

不是性能排名。而是：reasoning complexity。

11.3 Privatize First

例如 metrics：差：

```
32 workers
→ one global atomic counter
```

更好：

```
each worker
→ own counter
↓
periodic aggregation
```

这可以显著减少：

coherence traffic、atomic contention。

所以：**Privatize first, synchronize later.**

11.4 Single Writer Principle

单写者（single writer）使状态转换集中在一个序列中，减少多写者竞争和不变量证明分支。线程本地状态、按实体分片及消息传递，都可以用来构造这种所有权边界。单写者不意味着其他线程可无同步读取同一可变对象。读者仍需快照、锁、消息回复或经过论证的发布协议；消息本身也要说明转移后谁可以继续访问底层数据。

11.5 Message Passing 的真正价值

不是：“不用 shared memory。” Queue 本身仍然共享。真正变化的是：**Mutation Authority** 从：

```
Many writers
→ same state
```

变成：

```
Many producers
→ queue
→ one owner
→ state
```

11.6 Sharding

如果 single writer 成为 throughput ceiling：

```
key
↓
shard
↓
single owner
```

例如：

```
EntityId → shard 0
EntityId → shard 1
...
```

得到：

- within shard:
- single writer
- across shards:
- parallelism

11.7 Shard Key = Ownership Key

Sharding 的本质不是：“分几个 queue”。而是：哪个 key 决定谁拥有写入权限 (mutation authority)? 例如：

VehicleId、AccountId、PartitionId、ConnectionId。

选错 shard key：cross-shard coordination 会非常多。

11.8 Immutable Snapshot

不可变快照 (immutable snapshot) 把读者需要的一组字段作为一致版本发布，适合读多写少且允许读到稍旧版本的场景。读者取得快照所有权后不需要逐字段争锁，但发布与旧版本回收仍需协议。

```
std::atomic<std::shared_ptr<const Model>> current;
```

const Model 只约束这条访问路径，不自动证明深层对象不可变；若写者仍持有可变别名并同时修改内容，快照承诺便被破坏。shared_ptr 保证的是所管理对象的生命周期，不是所有关联状态的线程安全。

11.9 Snapshot Consistency vs Freshness

Reader 可能持有：V1 而 current 已经：V2 这可能是：stale but internally consistent。是否允许由 product/system contract 决定。不要把：latest 和：consistent generation 混为一谈。

11.10 Old Generation 的 Lifetime

not current ≠ safe to destroy

Old reader 可能还在使用 V1。因此仍需要：

shared_ptr、epoch、RCU。

等 generation reclamation。这和 lock-free node reclamation 完全同构。

12. 跨层关系图

12.1 Ownership Graph

所有权图标出对象的 owner、borrower、转移点与销毁者。同步边并不自动延长存活时间，所以线程开始工作之前，就要知道谁保证它访问的对象仍然存在。

12.2 Synchronization Graph

同步图标出每条 HB 的来源：读到发布的 acquire、相应 mutex 的解锁/加锁、线程完成后的 join 或已论证的队列交接。不能只画“线程 A → 线程 B”而省略触发该边的实际操作。

12.3 Mutation Graph

写入权限图（mutation authority graph）为每份状态列出全部写者、允许的读者与权限交接。多写者是重点审查位置，单写者也必须解释跨线程读取如何安全发生。

12.4 Progress Graph

进展图描述谁等待谁、谁有能力使谁继续，检查死锁、线程池饥饿和背压环。只有容量限制而没有释放容量的可执行路径，系统仍可能停住。

12.5 Lifetime / Reclamation Graph

生命周期与回收图区分逻辑移除和实际销毁。标出最后可能访问该对象的读者，以及证明这些访问已经结束的条件；不要把从容器 erase 当作跨线程的回收屏障。

12.6 Cache Ownership Graph

缓存所有权图追踪哪些核心高频写哪些物理邻近数据。它用于 G6 的争用/伪共享分析；race-free 仅是正确性前提，不等于一致性流量低。

12.7 Queue / Capacity Graph

容量图标出压力积累位置、满队列后的行为以及释放容量依赖谁。与进展图联读，可以发现 worker 全部阻塞在 submit、却无人能继续消费的闭环。

13. 统一并发审查

任何并发问题，建议按以下顺序。

Layer 1 — State

共享的 state 到底是什么？

Layer 2 — Ownership

- 谁拥有它?
- 谁控制 lifetime?

Layer 3 — Writers

- 有几个 writer?
- 能不能降到 1?

Layer 4 — Conflict

- read/read?
- read/write?
- write/write?

Layer 5 — Atomicity

- ordinary?
- atomic?
- compound invariant?

Layer 6 — Ordering

HB edge 从哪里来?

Layer 7 — Synchronization

选择:

- mutex?
- CV?
- atomic?
- CAS?
- queue?
- join?

Layer 8 — Lifetime

reader 持有 pointer 时 object 能不能死?

Layer 9 — Progress

- blocking?
- lock-free?
- wait-free?
- 谁会饿死?

Layer 10 — Backpressure

full / overload 时怎么办?

Layer 11 — Shutdown

- drain?
- abort?
- wake?
- join?

Layer 12 — Architecture

最后问：

- 这份 shared mutable state
- 真的需要共享修改吗？

这是最重要的一层。

14. 并发反模式

14.1 Smell 1 — Global Mutable State

```
all threads  
→ one global map/state
```

然后不断增加：

mutex、atomics、flags。

14.2 Smell 2 — Atomic Every Field

[机制片段 · 不承诺独立编译]

```
struct State {  
    std::atomic<int> a;  
    std::atomic<int> b;  
    std::atomic<int> c;  
};
```

但没人定义：multi-field consistency。

14.3 Smell 3 — Relaxed Cargo Cult

看到 atomic：

memory_order_relaxed

只因为：“更快”。却画不出 HB proof。

14.4 Smell 4 — Lock-free Cargo Cult

存在 mutex：第一反应就是 CAS。没有 profile，也没有 progress requirement。

14.5 Smell 5 — Detached Thread

```
std::thread{...}.detach();
```

但 thread lifetime、dependencies、shutdown 全不明确。

14.6 Smell 6 — Unbounded Queue

用：“不阻塞 producer”掩盖 processing capacity mismatch。

14.7 Smell 7 — Worker Waits Same Pool

Pool worker:

```
submit child  
↓  
wait child
```

没有分析 execution-capacity cycle。

14.8 Smell 8 — Notification as State

I got notify、therefore condition true。

错误。Predicate 才是 truth。

14.9 Smell 9 — Remove Then Delete

Lock-free structure:

```
CAS remove node  
↓  
delete immediately
```

却没有 reclamation proof。

14.10 Smell 10 — Shared shared_ptr<MutableT> Everywhere

共享拥有可以延长生命周期，却没有确定写入权限（mutation authority）。

14.11 Smell 11 — All Workers Touch All Entities

随机 dispatch 所有 state，造成：

locks、cache migration、poor affinity。

14.12 Smell 12 — Shutdown as One Bool

`bool running;`

却没有定义：

- `accepting?`
- `draining?`
- `aborting?`
- `stopped?`

15. 工程原则回查

如果半年后只能保留十五条：

1. 在讨论线程“看见什么”之前，先证明程序没有 **Data Race**。
2. **Happens-before** 是 C++ 并发推理的核心关系，不是 **wall-clock** 时间顺序。
3. **Atomicity**、**memory ordering**、**logical transaction** 是三个不同概念。
4. **Release/Acquire** 的本质是建立 **publication** → **consumption** 的 **synchronization relation**，不是刷新 **cache**。
5. **Mutex** 保护的是 **shared-state invariant**，而不是某个孤立变量。
6. **Condition variable** 等待的是 **predicate**；**notification** 只是唤醒提示。
7. **Atomic load + store** 不构成 **atomic state transition**；**CAS** 才能表达 **conditional commit**。
8. **CAS success** 不解决 **pointer lifetime**；**lock-free structure** 必须单独证明 **memory reclamation**。
9. **ABA** 的本质是 **current value** 恢复相同，但 **logical history/generation** 已改变。
10. **Lock-free** 是 **progress guarantee**，不是性能排名。
11. **SPSC/MPSC/SPMC/MPMC** 的区别本质上是 **writer/claimer ownership topology** 不同。
12. **Thread** 本身具有 **lifetime**；**shutdown** 必须遵循 **stop accepting** → **wake** → **drain/abort** → **exit** → **join** → **destroy**。
13. **Bounded queue/backpressure** 是系统稳定性 **contract**，不是容器细节。
14. 并发复杂度主要来自 **Shared Mutable State + Multiple Writers**，因此优先 **thread-local**、**single-writer**、**sharding**、**immutable snapshot** 和 **ownership transfer**。
15. 优秀并发架构的目标不是使用更聪明的 **synchronization**，而是让绝大多数业务 **state** 根本不需要 **concurrent mutation**。

16. 实验与验证

本章完整实验以 Markdown 中的源文件为准；从仓库根目录运行下列命令。执行器提取文件到新建临时目录，完整命令和原始输出写入结果记录，不修改历史制品。

[命令 · 自动提取、编译及分项记录]

```
python3 c++/learning/verify_handbook.py /usr/bin/clang++
└─ /opt/homebrew/opt/llvm/bin/clang++
```

只有带 `h-lab/h-file` 标记的完整实验及隔离反例参加本章定向执行；其他机制片段不是已验证的完整实现。改变条件用于理解判据，若未单独运行，不计作新增证据。

16.1 G7-D1 · 单次发布的不变量与动态观察

命题、观察与边界。每轮创建新的 payload 和 ready，写者只写一次；读者只在 acquire 观察到发布后读取。写入 SB release、release SW acquire、acquire SB 读取给出 HB 链。join 完成后才销毁状态；有限 stress 与 TSan 无诊断仅支持本次路径，不证明所有调度、公平性或复用协议。

[完整实验 · G7-D1 · main.cpp]

```
#include <atomic>
#include <iostream>
#include <thread>

int main() {
    for (int round = 0; round < 200; ++round) {
        int payload = 0;
        std::atomic<bool> ready{false};
        bool correct = false;
        std::thread consumer([&] {
            while (!ready.load(std::memory_order_acquire))
                std::this_thread::yield();
            correct = (payload == 42);
        });
        std::thread producer([&] {
            payload = 42;
            ready.store(true, std::memory_order_release);
        });
        producer.join();
        consumer.join();
        if (!correct) return 1;
    }
    std::cout << "publication=200" << std::endl;
}
```

运行与判据。上述统一命令中的 G7-D1 提取并处理本模块。先运行普通构建，再在 TSan 可用时运行插桩构建；需要指定输出和正常退出，超时为失败而非停机成功。

16.2 G7-D2 · 有界通道：排空、拒绝与停机

命题、观察与边界。队列和 closed 由同一 mutex 保护；push 插入与 close 置位是各自线性化点。pop 在 closed 且空时结束，否则按 FIFO 取出；close 后不接受新值，已接受值仍排空。notify 只负责唤醒，谓词决定动作。实验检查完整 FIFO、空通道关闭、满通道关闭及重复 close；等待中的计数器在同锁下登记，保证关闭确实发生在等待路径，不用 sleep 猜时序。

[完整实验 · G7-D2 · main.cpp]

```
#include <condition_variable>
#include <deque>
#include <iostream>
#include <mutex>
#include <optional>
#include <thread>

class Channel {
    std::mutex mutex_;
    std::condition_variable changed_;
    std::deque<int> queue_;
    bool closed_ = false;
    unsigned waiting_ = 0;
public:
    bool push(int value) {
        std::unique_lock lock{mutex_};
        while (!closed_ && queue_.size() == 4) {
            ++waiting_; changed_.notify_all();
            changed_.wait(lock);
            --waiting_;
        }
        if (closed_) return false;
        queue_.push_back(value);
        changed_.notify_all();
        return true;
    }
    std::optional<int> pop() {
        std::unique_lock lock{mutex_};
        while (!closed_ && queue_.empty()) {
            ++waiting_; changed_.notify_all();
            changed_.wait(lock);
            --waiting_;
        }
        if (queue_.empty()) return std::nullopt;
        int value = queue_.front();
        queue_.pop_front();
        changed_.notify_all();
        return value;
    }
    void close() {
```

完整实验 · G7-D2 · main.cpp · continued

```

        std::lock_guard lock{mutex_};
        closed_ = true;
        changed_.notify_all();
    }
    void wait_until_blocked() { // Test observation, not a production API.
        std::unique_lock lock{mutex_};
        changed_.wait(lock, [&] { return waiting_ != 0; });
    }
};

int main() {
    for (int round = 0; round < 50; ++round) {
        Channel channel;
        bool producer_ok = true, consumer_ok = true;
        int count = 0;
        std::thread reader([&] {
            while (auto value = channel.pop()) {
                if (*value != count) consumer_ok = false;
                ++count;
            }
        });
        std::thread writer([&] {
            for (int i = 0; i < 1000; ++i)
                if (!channel.push(i)) producer_ok = false;
            channel.close();
        });
        writer.join();
        reader.join();
        channel.close();
        if (!producer_ok || !consumer_ok || count != 1000 ||
            channel.push(1001) || channel.pop()) return 1;
    }
    Channel empty;
    bool ended = false;
    std::thread reader([&] { ended = !empty.pop(); });
    empty.wait_until_blocked();
    empty.close();
    reader.join();
    if (!ended) return 2;

    Channel full;
    for (int i = 0; i < 4; ++i) if (!full.push(i)) return 3;
    bool rejected = false;
    std::thread writer([&] { rejected = !full.push(4); });
    full.wait_until_blocked();
    full.close();
    writer.join();
    if (!rejected) return 4;
}

```

完整实验 · G7-D2 · main.cpp · continued

```

    for (int i = 0; i < 4; ++i) {
        auto value = full.pop();
        if (!value || *value != i) return 5;
    }
    if (full.pop()) return 6;
    std::cout << "fifo=50000; empty-close; full-close; joined" << std::endl;
}

```

运行与判据。上述统一命令中的 G7-D2 提取并处理本模块。先运行普通构建，再在 TSan 可用时运行插桩构建；需要指定输出和正常退出，超时为失败而非停机成功。

16.3 G7-D3 · 工具阳性对照：缺少同步的写冲突

命题、观察与边界。反例有意让两个线程无同步写入同一普通对象。只在隔离进程中启用 TSan 运行，要求 data race 诊断和指定退出码；不能把结果值、普通崩溃或超时算作成功。它验证工具对这个已知错误的检测能力，不验证 G7-D1/D2 的所有路径。

[反例 · 未定义行为：仅限 TSan 隔离检测 · G7-D3 · main.cpp]

```

#include <thread>
int value = 0;
int main() {
    std::thread writer([] { value = 1; });
    value = 2; // Deliberate data race: no HB edge between the two writes.
    writer.join();
}

```

运行与判据。上述统一命令中的 G7-D3 提取并处理本模块。先运行无竞争的 TSan 探针；可用时要求 data race 诊断与退出码 66。环境不可用明确 SKIP，不将普通崩溃充作检测。**协议论证边界。**D2 假设 mutex/CV 与标准容器正确、工作线程获得调度、内存分配不失败；它不覆盖任务执行异常、强制终止、MPMC 无锁回收或 stop-token API。状态只在锁内转换，所有工作线程先 join 再销毁 channel。有限 stress/TSan 结果不能证明公平性或全程序无竞争；D3 阳性对照也不能提升这一结论。

17. Final Gate

下面这些问题应该能闭卷回答。

A. Data Race

1 为什么：

[机制片段 · 不承诺独立编译]

```
int x = 0;

// A
x = 1;

// B
use(x);
```

在没有 synchronization 时不能只说：“B 可能读到 0 或 1”？

2 为什么：sleep 1 second 不能建立 happens-before？

B. Happens-before

3 解释：

SB + SW + transitivity $\rightarrow$ HB

4 为什么普通 payload 可以通过 atomic flag publication 安全跨线程？

C. Memory Order

5 什么场景适合：

memory_order_relaxed

？

6 为什么：

[机制片段 · 不承诺独立编译]

```
data = 42;
ready.store(true, relaxed);
```

不能配合 relaxed load 正确发布普通 data？

7 acq_rel 为什么主要自然出现在 RMW 上？

8 seq_cst 比 release/acquire 多提供的核心是什么？

D. Mutex / CV

9 为什么 mutex 保护的是 invariant 而不是 variable？

10 为什么：

[机制片段 · 不承诺独立编译]

```
if (!pred()) {
    cv.wait(lock);
}
```

通常错误？

11 为什么 notification 可以发生在 waiter 真正等待之前，而正确 predicate-based design 仍然不丢工作？

E. CAS

12 CAS failure 为什么修改 expected？

13 为什么：weak CAS 不等于：weak memory ordering？

14 为什么 atomic：

load + store

不能替代 CAS？

F. ABA / Reclamation

15 解释：

$A \rightarrow B \rightarrow A$

为什么 CAS 可能无法发现中间变化。

16 为什么：

Unlinked $\neq$ Safe to delete

？

17 Hazard Pointer 与 Epoch 的核心区别是什么？

18 为什么 generation/tagged pointer 不能自动解决 lifetime reclamation？

G. Queue

19 SPSC 为什么通常不需要 CAS 更新 head/tail？

20 SPSC 为什么需要两个方向的 release/acquire handoff？

21 MPSC 中为什么：reservation tail 不能直接等价于：published tail？

22 Per-slot sequence number 在 MPMC queue 中解决的核心是什么？

H. Thread Lifetime

23 为什么 joinable `std::thread` 析构会 terminate？

24 为什么 detach 不能解决 object lifetime？

25 为什么 `jthread.request_stop()` 不能强制杀死 thread？

26 为什么 thread member 通常应该最后声明？

I. Thread Pool

27 为什么 worker 不应该持 queue mutex 执行 Task？

28 为什么 bounded pool 中 worker 递归 blocking submit 可能 deadlock？

29 为什么 CPU-bound 与 blocking-I/O task 不应该机械使用同一个 pool？

30 为什么 work stealing 的主要架构价值不是“更复杂”，而是 common-case local ownership?

J. Architecture

31 为什么 single writer 能显著降低 concurrency complexity?

32 Sharding 的本质是什么? 不是: 多几个 queue。而是?

33 Immutable snapshot 主要优化什么 workload?

34 为什么: old snapshot no longer current 仍不代表可立即销毁?

35 为什么“每个字段都 atomic”经常不如一个 immutable snapshot?

18. Final Gate · 参考答案与常见误判

18.1 Data Race、HB 与内存序 (1 ~ 8)

1. 无同步的冲突非原子访问属于数据竞争，语言不把结果限定为旧值/新值二选一。还必须验证对象生命周期，而不只检查 load/store。
2. sleep 改变调度概率，不建立 synchronizes-with；必须由锁、原子发布或其他标准规定的同步机制建立关系。
3. 在线程内用 sequenced-before 排列相关操作，跨线程找出 synchronizes-with，再使用传递性得到所需 HB。箭头必须有规则依据，不能按墙钟时间补画。
4. 非原子 payload 的写与读可由单次 release/acquire 发布建立 HB。读者必须读到相应发布；写者不能在读者使用时再次改写同一 payload。
5. 独立计数且不依赖该计数发布其他数据时可使用 relaxed；还要分析计数溢出、复合不变量和对象存活。
6. relaxed 的读写没有所需发布同步，标志观察本身不使普通 data 读写合法。
7. RMW 既读取此前状态又发布本次更新，可能同时需要 acquire 和 release；纯 load/store 不需要也不接受所有 RMW 组合。
8. seq_cst 提供对相应 SC 操作的受约束总序，不会把其他错误协议变正确；混合弱序操作须另行推理。

18.2 Mutex、CV 与 CAS (9 ~ 14)

9. 相关字段组成一个状态不变量；只给各字段分别上锁可能仍让读者观察到不允许的组合。
10. wait 可伪唤醒，醒来后谓词也可能已被其他线程改变；使用循环或谓词重载，并在同一锁协议下检查状态。
11. 不是因为通知被保存，而是谓词状态受锁保护并持久存在。消费者持锁检查和 wait 的原子解锁/等待配合生产者的持锁修改，消除丢失工作的窗口；锁外 atomic 标志加 notify 不能机械替代。
12. 失败意味着当前值不同或 weak 伪失败；expected 接收观察结果，下一次 desired 应由它重算。不能反复用旧假设提交状态。
13. weak 指允许伪失败，内存序指定同步/排序，二者独立。
14. 独立原子 load 和 store 之间可插入另一修改；CAS 把条件检查与更新合成一个原子转换。

18.3 ABA、回收与队列 (15 ~ 22)

15. 比较当前值表示不能看到 A→B→A 的历史，因此相同表示不保证相同代次。
16. 从结构摘除只阻止某些新获取，不能证明此前读者已不再使用对象。
17. Hazard pointer 公开保护具体对象并要求正确的获取/重验协议；epoch 等待相关读者离开旧时期后批量回收。读者停顿、注册成本与回收延迟的权衡不同。
18. tag 可区分某些历史变化，却不阻止对象被释放；还要处理回绕并提供真正的存活保护。
19. 常见 SPSC 中每个游标各有唯一写者，不需要多个写者竞争同一个增量；交叉观察仍要同步。

20. 正向发布元素供消费，反向发布消费完成供生产者安全复用槽位；缺少反向边会让下一轮写覆盖仍在读取的数据。
21. 取得位置不等于元素已经构造完成；多生产者可按不同速度完成，消费者不能跨过未发布位置。
22. 常见有界环中每槽序号区分空闲、就绪与复用代次；这只是协议组成，不是所有 MPMC 算法或内存回收的通用证明。

18.4 线程生命周期与运行时 (23 ~ 30)

23. joinable thread 的析构按标准调用 `terminate`；不能偷偷把仍运行的工作和依赖责任丢掉。
24. `detach` 只分离线程句柄的等待责任，不延长捕获对象生命周期，也不建立停机确认。
25. `stop_token` 是合作机制；工作函数与阻塞操作必须响应请求。不能依靠强杀跳过不变量恢复。
26. 最后声明通常先析构，可使 `jthread` 先于依赖销毁；仍须满足唤醒、退出、`join` 和锁依赖，声明顺序不是完整方案。
27. 在队列锁内执行任务会串行化调度，并可能被回调重入、阻塞和再次提交拖入死锁。
28. 所有 `worker` 都等待有界队列空位时，可能已没有线程能消费腾出位置。需非阻塞/拒绝、`helping` 或不同依赖拓扑。
29. 阻塞 I/O 占住执行槽，会饿死需要 CPU 或完成回调的工作；线程数、队列和资源隔离须按负载决定。
30. `Work stealing` 保留常见路径的本地队列操作，仅在需要时跨线程协调；仍有窃取、任务依赖与停机复杂度，并非自动更快。

18.5 架构 (31 ~ 35)

31. 单写者集中状态转换和不变量；其他线程的读取仍需同步或快照，不能据此无锁读取可变状态。
32. 分片按稳定的 `ownership key` 划分写入权限、执行位置与关联数据；跨片事务、迁移和热点仍须协议，不只是多建几个队列。
33. 不可变快照适合读多写少、允许一定陈旧度的负载，以发布和版本保留换取简单读取。
34. “非当前”不等于无人持有；必须等全部相关读者释放或到达回收安全点。
35. 各字段 `atomic` 不自动构成一致版本。快照把相关字段作为一个逻辑版本发布，但需要真正不可变及正确生命周期管理。

19. G0 ~ G7 的统一模型

下面只列跨章审查的交接点，概念定义回到各章主讲处；本节保留它们在并发问题中的用途，不另写一套定义。

19.1 G1 — Object Model

由 G1 §1 ~ 2 检查存储、生命周期、类型、边界与对齐，先确认访问合法，再讨论同步。

19.2 G2 — Ownership

由 G2 §1、§9 列出拥有者、借用者、交接与清理；并发新增的问题是使用者是否真的都已退出。

19.3 G3 — Value Semantics

由 G3 §4、§8 区分复制、移动、分配与表示，不由指针大小推断跨线程交接的完整成本。

19.4 G5 — Genericity

由 G5 §9 界定哪些变化留在编译期，避免调度与生命周期策略无必要地扩散成模板参数。

19.5 G6 — Performance

由 G6 §8 ~ 9 分析布局、缓存、分配、TLB、分支和一致性成本；这些观察不替代本章的语言顺序论证。

19.6 G7 — Concurrency

- Who may access or mutate this state concurrently,
- and what ordering/lifetime guarantees make that legal?

最终统一：

```
Object
  ↓
Lifetime
  ↓
Ownership
  ↓
Representation
  ↓
Machine Cost
  ↓
Mutation Authority
  ↓
Synchronization
  ↓
Concurrent Architecture
```

这已经是一套相当完整的 systems reasoning model。

20. 实际代码的跨层审查

看到：

[机制片段 · 不承诺独立编译]

```
std::shared_ptr<MutableModel> model;
std::mutex mutex;

void worker() {
    std::lock_guard lock{mutex};
    model->update(...);
}
```

以前可能只问：“mutex 会不会慢？”现在应该按完整链路分析：

G1

model pointee lifetime 是否有效？

G2

为什么是 shared ownership？

G3

shared_ptr copy/refcount 成本是什么？

G6

model working set 是否在 cores 间迁移？

mutex/cache-line 是否 contention？

G7

为什么有多个 writers？

能否 single writer？

能否 shard by model/entity？

readers 是否可以使用 immutable snapshot？

这项分析的目的在于：把 **synchronization problem** 向上提升成 **ownership architecture problem**。

21. 本章范围与后续阅读

数据竞争、内存序、锁与等待、CAS、回收、队列、线程停机、任务运行时和并发架构均保留为本章主题。协议论证是明确假设下的工程推理，不是全程序形式化验证。下一阶段按总导航进入 [G8 ABI](#) 与 [C 互操作](#)；接口一旦交给独立组件，线程与销毁合同也必须穿过边界。需要完整关闭案例时回查 [G10 §7](#)。

22. 参考与验证入口

全系列导航 · 实验说明 · 本批修订与证据

语言模型参见 [N4950 数据竞争、原子内存序](#)；工具边界参见 [Clang ThreadSanitizer](#)。